

1 Report2

1.1 回文数组

1.1.1 形式化定义

将一个数字序列经过若干次操作后变成回文序列, 求出将该数组变为回文数组所需的最小操作次数。操作支持如下:

- 选择数组中相邻的两个数, 同时将其加 1 或减 1;
- 选择数组中单独的一个数, 将其加 1 或减 1。

1.1.2 解题思路

联想这道题的解题思路: <https://www.luogu.com.cn/problem/P5019>

我们可以想到贪心的思路, 上面那道题中, 我们可以靠前一个坑顺便填充下一个坑, 这道题我们可以发现第 1 个操作就是一个顺便的操作。所以我们记录好每个位置的对称差, 再让前一个对称差能够顺便填充后一个对称差即可。不过需要注意的是对称差是有正负的, 和坑洞填充只有一个方向是不同的。

1.1.3 完整代码

```
1  #include <iostream>
2  #include <cstdio>
3  #include <cmath>
4  #define ll long long
5  using namespace std;
6
7  const int N = 2e5 + 10;
8
9  int main() {
10     int n;
11     scanf("%d", &n);
12     int a[N];
13     for (int i = 1; i <= n; ++i) {
14         scanf("%d", &a[i]);
15     }
16
17     ll tot = 0;
18     for (int i = 1; i <= n / 2; ++i) {
19         int diff = a[i] - a[n - i + 1];
20         tot += abs(diff);
21
22         if (i + 1 <= n / 2) {
```

```

23         int next_diff = a[i + 1] - a[n - i];
24         if ((diff >= 0 && next_diff >= 0) || (diff <= 0 && next_diff <= 0)) {
25             if (abs(diff) >= abs(next_diff)) {
26                 a[i + 1] = a[n - i];
27             } else {
28                 a[i + 1] -= diff;
29             }
30         }
31     }
32 }
33 printf("%lld", tot);
34 return 0;
35 }

```

1.2 平衡饮食

1.2.1 形式化定义

背包的容量为 X ，其中要装入 3 种物品，每种物品有其对应的价值 A_i 和体积 C_i ，现在希望你采取合适的装包方式，使得 3 种物品各自的总价值 F_1, F_2, F_3 的最小值最大。即，使得 $\min(F_1, F_2, F_3)$ 最大。

1.2.2 解题思路

采用动态规划和二分答案的思想。

- 状态定义： $dp[i][j]$ ，种类 i 用 j 容量所能获得的价值最大值
- 状态转移方程： $dp[i][j] = \max(dp[i][j], dp[i][j - c] + a)$ ，对每种食物，需要逆向更新数组，经典的 01 背包问题。
- 初始条件： $dp[i][0] = 0$ ，其他均设置为 INF ，表示该情况非法。
- 前缀最大值优化： $dp[i][j] = \max(dp[i][j], dp[i - 1][j])$ ，如果当前容量 j 下，无法获得价值，则直接取上一步的最大值。

对于本题，显然是满足单调性的，如果可以获取更大的最低价值，则一定可以获取更小的最低价值。所以满足二分答案的条件。即，将优化问题转化为判定问题。对于给定的目标值 mid ，判断是否存在一个组合可以使得在满足容量限制的情况下，3 种物品的总价值均大于 mid 。这样就能说明答案至少是 mid ，可以尝试更大的答案。

Listing 1: 二分答案部分代码

```

1 // 二分查找
2 int low=0, high=min({dp[1][x], dp[2][x], dp[3][x]});
3 while(low <= high) {

```

```

4     int mid = (low+high)/2;
5     if(可以满足min(V1,V2,V3)>=mid)
6         low = mid+1;
7     else
8         high = mid-1;
9 }
10
11 // 二分验证mid
12 int sum = 0, ok = 1;
13 for (int v = 1; v <= 3; v++) {
14     int cost = lower_bound(dp[v], dp[v] + x + 1, mid) - dp[v];
15     if ((sum += cost) > x || cost > x) {
16         ok = 0; break;
17     }
18 }

```

1.2.3 复杂度分析

对于状态转移部分，每个物品都要遍历 $x - c$ 次，总共 n 个物品，所以状态函数计算部分的复杂度为：

$$T_{DP} = O(n \cdot x)$$

二分答案对于验证部分需要对 3 种物品进行二分查找：

$$T_{check} = O(3 \cdot \log x) = O(\log x)$$

设物品中最大的价值为 V_{max} ，则二分的次数为：

$$T_{search} = O(\log V_{max})$$

所以总时间复杂度为：

$$T_{total} = T_{DP} + T_{check} \cdot T_{search} = O(n \cdot x) + O(\log x \cdot \log V_{max})$$

1.2.4 完整代码

```

1  #include <iostream>
2  #include <cstdio>
3  #include <algorithm>
4  #define int long long
5  using namespace std;
6
7  const int INF = 1e9;
8  const int MX = 5e3+5;

```

```

9
10 int dp[4][MX]; // dp[v][c]: 维生素v用c卡路里能获得的最大量
11 int n, x;
12
13 int max(int a, int b){
14     return a>b ? a : b;
15 }
16
17 signed main() {
18     // freopen("in.txt", "r", stdin);
19     scanf("%lld%lld", &n, &x);
20
21     for(int v=1; v<=3; v++) {
22         fill(dp[v], dp[v]+x+1, -INF);
23         dp[v][0] = 0;
24     }
25
26     for(int i=1; i<=n; i++) {
27         int v, a, c;
28         scanf("%lld%lld%lld", &v, &a, &c);
29         for(int j=x; j>=c; j--) {
30             if(dp[v][j-c] != -INF) {
31                 dp[v][j] = max(dp[v][j], dp[v][j-c]+a);
32             }
33         }
34     }
35
36     for(int v=1; v<=3; v++) {
37         for(int j=1; j<=x; j++) {
38             dp[v][j] = max(dp[v][j], dp[v][j-1]);
39         }
40     }
41
42     int low = 0, high = min({dp[1][x], dp[2][x], dp[3][x]}), ans = 0;
43     if(high < 0) return printf("0"), 0;
44
45     while(low <= high) {
46         int mid = (low+high)>>1;
47         int sum = 0, ok = 1;
48
49         for(int v=1; v<=3; v++) {

```

```

50         int* d = dp[v];
51         int cost = lower_bound(d, d+x+1, mid) - d;
52         if((sum += cost) > x || cost > x) {
53             ok = 0; break;
54         }
55     }
56
57     ok ? (ans=mid, low=mid+1) : (high=mid-1);
58 }
59 printf("%lld", ans);
60 }

```

1.3 其实 Allen 是幕后大 BOSS

1.3.1 形式化定义

给定一个长度为 n (n 为奇数) 的员工序列, 其中第 i 个员工的工资范围为 $[l_i, r_i]$ 。初始总工资满足 $\sum_{i=1}^n l_i \leq s$ 。我们需要为每个员工分配工资 $w_i \in [l_i, r_i]$, 满足:

1. 总工资约束: $\sum_{i=1}^n w_i \leq s$
2. 最大化工资序列的中位数

设 $k = \frac{n+1}{2}$, 优化目标为:

$$\max \{\text{median}(w_1, w_2, \dots, w_n)\}$$

约束条件:

$$\begin{cases} l_i \leq w_i \leq r_i & \forall i \in \{1, 2, \dots, n\} \\ \sum_{i=1}^n w_i \leq s \end{cases}$$

1.3.2 解题思路

先发出基础工资, 把每位员工工资初始化为 $l[i]$, 再看此时高于设定目标中位数 mid 的员工数量, 若达到要求就返回, 否则调整后续员工工资; 为了调整中位数但尽可能小的增加总工资, 应该优先调整**最接近中位数的元素**, 所以综合考虑前面的论述, 我们必须要对员工的最低工资 $l[i]$ 进行**从大到小**排序。

当目标中位数 mid 可以达成时, 则挑战更高的中位数标准, 若不行, 则尝试更低的中位数标准。需要注意的是, **每次尝试之间需要把总工资回调到原来的状态**, 这里我们并没有对每个人的具体工资做改动, 而是只改动总工资, 在效果上相当于改动了每个人的具体工资。

1.3.3 复杂度分析

- 预处理: $O(n)$ 计算初始总工资
- 排序: $O(n \log n)$

- 二分查找: $O(\log(\max r_i))$
- 每次检查: $O(n)$
- 总复杂度: $O(n \log n + n \log(\max r_i))$

1.3.4 完整代码

```

1  #include <bits/stdc++.h>
2  #define ll long long
3  using namespace std;
4
5  const int N = 2e5 + 10;
6  int t;
7  int n;
8  ll s;
9  int l[N], r[N];
10 ll ans;
11 ll minl=2e14, maxr=0;
12
13
14 ll min(ll a, ll b){
15     return a < b ? a : b;
16 }
17 ll max(ll a, ll b){
18     return a > b ? a : b;
19 }
20 bool cmp(int a, int b) {
21     return l[a] > l[b];
22 }
23 void solve() {
24     ll sum_li = 0;
25     for (int i = 1; i <= n; ++i) {
26         sum_li += l[i];
27     }
28
29     vector<int> idx(n);
30     for (int i = 0; i < n; ++i) {
31         idx[i] = i + 1;
32     }
33     sort(idx.begin(), idx.end(), cmp);
34

```

```

35     int k = (n + 1) / 2;
36     ll left = minl, right = maxr;
37     ans = 0;
38
39     while (left <= right) {
40         ll mid = left + (right - left) / 2;
41         ll current_sum = sum_li;
42         int cnt = 0;
43         int i = 0;
44
45         // 高工资统计
46         for (; i < idx.size(); ++i) {
47             int id = idx[i];
48             if (l[id] > mid) {
49                 cnt++;
50                 if (cnt == k) break;
51             } else {
52                 break;
53             }
54         }
55
56         // 高工资数量足够多，直接尝试下一种可能
57         if (cnt >= k) {
58             ans = mid;
59             left = mid + 1;
60             continue;
61         }
62
63         // 调整工资
64         for (; i < idx.size(); ++i) {
65             int id = idx[i];
66             if (r[id] >= mid) {
67                 current_sum += (mid - l[id]);
68                 cnt++;
69                 if (cnt == k) break;
70             }
71         }
72
73         if (cnt >= k && current_sum <= s) {
74             ans = mid;
75             left = mid + 1;

```

```

76         } else {
77             right = mid - 1;
78         }
79     }
80 }
81
82 int main() {
83     // freopen("in.txt", "r", stdin);
84     scanf("%d", &t);
85     while (t--) {
86         ans = 0;
87         scanf("%d%lld", &n, &s);
88         for (int i = 1; i <= n; ++i) {
89             scanf("%d%d", &l[i], &r[i]);
90             minl = min(minl, l[i]);
91             maxr = max(maxr, r[i]);
92         }
93         solve();
94         printf("%lld\n", ans);
95     }
96     return 0;
97 }

```

1.4 Sherway 与幂

1.4.1 形式化定义

求 1 到 N 中共有多少个数可以表示成 $M^K, K > 1, N \leq 1e18$.

1.4.2 解题思路

我们会先有一个朴素的思想，也就是我对 n 进行开根，把开根的结果取整后加起来：

$$ans = \sum_{i=2}^{\infty} \lfloor (n)^{\frac{1}{i}} \rfloor$$

但是这样我们发现，会对个别项重复计算，例如 $2^6 = 4^3 = 8^2 = 64$ 被算了 3 次，我们应该对其进行去重。这样我们就应该联想到容斥原理来确保不会重复计算，也不会额外扣除。

设 A_i 表示所有 $p[i]$ 次方数的集合 ($p[i]$ 表示第 i 个质数)，即：

$$A_i = \{x^{p[i]} \leq n | x \in \mathbb{N}\}$$

我们需要计算的就是 $|\bigcup_{i=1}^{\infty} A_i|$ ，即所有次方数的并集。所以有：

$$|\bigcup_{i=1}^{\infty} A_i| = \sum_{i=1}^{\infty} |A_i| - \sum_{1 \leq i < j} |A_i \cup A_j| + \sum_{1 \leq i < j < k} |A_i \cup A_j \cup A_k| \dots$$

为了计算上式，所以我们首先需要确定指数取值的上界，只需要找到 $imax$ 使得 $2^{p[imax]} \leq n < 2^{p[imax+1]}$ 即可。接着在中间的计算过程中，对于由奇数个质数相乘得到的指数幂集合应该加上，而对于偶数质数相乘得到的指数幂集合应该减去。这也都是我们需要统计的。

需要额外注意的是，这里我们可以利用已求出的 $imax$ ，那么从 1 到 2^{imax} 中的每一个 j ，它的二进制表示可以算作是对于素数数组的一个选取规则。例如 0b101 可以表示选择了素数 2 和 5。这样只需要记录其中 1 的数量，就知道其选取的素数的奇偶数了。

1.4.3 复杂度分析

- 外层循环： $2^{imax} - 1$ 次
- 内层循环 $imax$ 次
- 总复杂度 $O(imax \cdot (2^{imax} - 1))$ (其中 $imax$ 约为 2 到 $\log n$ 中质数的数量，在本题中不超过 20)

1.4.4 完整代码

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  #define ll long long
5  ll n, ans;
6
7  long double ONE = 1.0;
8
9  // 由于n最大是1e18, 所以不会超过1 << 67
10 int ps[] = {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67};
11
12 void solve() {
13     int max_exp = 0;
14     while(1ll << ps[max_exp + 1] < n) max_exp++;
15
16     for (int mk = 1; mk < (1 << max_exp); mk++) {
17         ll exp = 1;
18         bool flag = false;
19
20         for (int j = 0; j < max_exp; j++) {
21             if (mk & (1 << j)) {
22                 exp *= ps[j];
23                 flag = !flag;
24             }
25         }
26         // 特别注意此处对于浮点数精度的调整。
```

```

27     long double one = 1;
28     ll cnt = (ll)(powl(n, one / exp)+1e-10);
29
30     if (flag) {
31         ans += cnt;
32     } else {
33         ans -= cnt;
34     }
35 }
36 }
37
38 int main() {
39     scanf("%lld", &n);
40     solve();
41     printf("%lld\n", ans + 1); // 考虑到1一定满足题意，而前面没有统计。
42     return 0;
43 }

```